

Week 5

PyTorch Foundations. Tensors and Data Structures

- Introduction to PyTorch for ML and AI workflows
- Key data structures for deep learning
- Foundation for all future model development

Learning Objectives

- Understand the role of PyTorch in industry-standard ML workflows
- Create and manipulate tensors using PyTorch
- Compare PyTorch with NumPy and discuss industry relevance
- Interpret PyTorch documentation and apply best coding practices
- Prepare for your first PyTorch assessment project

Agenda

1. Where are we in the course sequence?
2. What is PyTorch and why is it used in industry?
3. Comparing PyTorch and NumPy for ML tasks
4. Tensors. Foundation of PyTorch
5. Core tensor operations; hands-on practice
6. PyTorch data structures; DataLoader and datasets
7. Reading documentation and coding best practices
8. Assessment connections and activity

Where Are We?

Review of Weeks 1-4

- Bash and scripting basics for Linux/cloud environments
- Designing logic for automating ML workflows
- Coding standards and version control for collaborative projects
- This week. Transition from foundational scripting to industry ML tools

Why PyTorch?

Industry and TAFE Relevance

- Preferred library for deep learning and AI research
- Built-in support for GPU acceleration; scalable to cloud/HPC
- Pythonic interface; well-documented and widely used in ML engineering jobs
- Used in local projects and industry partnerships

PyTorch vs. NumPy

Practical Differences

Feature	NumPy	PyTorch
Data Structure	ndarray	Tensor
GPU Support	No	Yes (easy CUDA support)
Autograd	No	Yes (auto differentiation)
Used for	General computation	ML and DL workflows
Exports to ONNX	Not directly	Yes

- PyTorch is built on concepts from NumPy, but adds ML, GPU, and automation features

What is a Tensor?

Core Concept for ML

- Generalization of arrays and matrices for machine learning
- Support for N-dimensional data; efficient storage and computation
- Enables both batch data processing and model parameter representation
- Essential for expressing all key ML operations

Creating and Manipulating Tensors

- Use Python and PyTorch to create, view, and transform tensors
- Example; Creating a tensor from a Python list

```
import torch
t = torch.tensor([1, 2, 3, 4])
print(t)
```

- Check the shape, data type, and device (CPU or GPU)

```
print(t.shape)
print(t.dtype)
```

- Move tensor to GPU if available

```
t = t.cuda() # Only works if GPU and CUDA available
```


Hands-On; Basic Tensor Operations

Try these on your own or with a partner:

- Create two 2D tensors with random numbers
- Add, subtract, and multiply them; print the results
- Reshape a tensor using `.view()` or `.reshape()`
- Stack tensors along different dimensions
- Convert a tensor to a NumPy array and back

Key Data Structures in PyTorch

- Tensor; main data structure for all computation
- Dataset; interface for loading and managing data
- DataLoader; handles batching, shuffling, parallel data loading
- Module; base class for all neural networks
- Example. Loading data for training

```
from torch.utils.data import DataLoader, TensorDataset
dataset = TensorDataset(torch.randn(100, 4), torch.zeros(100))
loader = DataLoader(dataset, batch_size=10, shuffle=True)
```

Interpreting PyTorch Documentation

- PyTorch docs: <https://pytorch.org/docs/stable/index.html>
- Use function signatures, examples, and parameter lists
- Pay attention to device management; CPU vs GPU
- Read usage notes and see real-world patterns in code samples

Best Practices for ML Code in PyTorch

- Use version control for all scripts (see Week 4)
- Structure code with functions and modules; avoid repetition
- Document shapes, data types, and device assumptions
- Follow naming and commenting standards from previous weeks
- Test key tensor operations before integrating into pipelines

Industry Use Case; Tensors in HPC Workflows

- Large-scale model training uses tensor operations across thousands of cores
- PyTorch enables parallel GPU computation, vital for enterprise and cloud-scale AI
- Example. Government agency using PyTorch on Azure to process satellite data in parallel
- Skills with tensors transfer directly to industry HPC and MLOps environments

Activity; Practice and Apply

- In pairs; create sample tensors to represent "images" (e.g. batch of 10, 3x32x32)
- Experiment with slicing and transforming data
- Document your code following the guidelines from Week 4
- Discuss. Why is understanding tensor shapes critical in ML projects?

Assessment Preparation

- This week covers first half of PyTorch fundamentals for your assessment portfolio
- Practice; upload code samples showing tensor creation and manipulation
- Ensure you can move tensors to GPU, reshape them, and perform basic operations
- Check; Can you explain the difference between a NumPy array and a PyTorch tensor?

Summary and Next Steps

- PyTorch tensors are the core building blocks for all AI/ML models
- Industry demand; skills in efficient data representation and manipulation
- Next week; go deeper into PyTorch with autograd and neural network foundations
- Start drafting your portfolio with today's hands-on code and reflections

